به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین سوم

مدرس: مهدی صفرنژاد — انتشار: ۲۰ اسفند ۹۷

درخت‌ها

سؤال ۱. **[۲۰ نمره]** در این سوال باید تعدادی عمل درج و حذف روی دو داده‌ساختار پرکاربرد انجام دهید. برای دیدن نحوهٔ درج و حذف در درخت قرمز–سیاه می‌توانید به اینجا و برای دیدن درج و حذف در هرم کمینه به اینجا مراجعه کنید. دنباله‌ای از درج و حذف‌ها در ادامه آمده است. به طور خلاصه اعمال صورت گرفته (مثلاً دوران‌ها) را شرح دهید.

آ. **[۱۰ نمره]** در دنباله زیر، علامت $+$ به معنای درج و $-$ به معنای حذف است. دنباله اعمال زیر را به ترتیب از چپ به راست روی درخت قرمز–سیاه اعمال کنید:

$$+12, +5, +7, +6, +2, +1, +9, -5, -7, +3, -12, +10, +11, -6, -2, -10$$

ب. **[۱۰ نمره]** دنباله اعمال زیر را روی داده‌ساختار هرم **کمینه** به ترتیب از راست به چپ اعمال کنید (به کمینه بودن هرم توجه کنید):

$$+10,\ +6,\ +2,\ +4,\ \text{delete-min},\ +8,\ \text{delete-min},\ +12,\ +3$$

پاسخ: در مراجع گفته شده می‌توان دنباله‌های عملیات گفته شده را اجرا کرد و پاسخ مسئله را به صورت تصویری مشاهده کرد.

سؤال ۲. **[۲۵ نمره]** برای هر یک از مسائل زیر داده‌ساختار مناسبی پیشنهاد دهید و انتخاب خود را به طور مختصر توجیه کنید. برای این کار می‌توانید از ترکیب داده‌ساختارهایی که خوانده‌اید نیز استفاده کنید. در این صورت داده‌ساختار خود را شرح داده و آن را از نظر زمانی تحلیل کنید. توجه کنید که حافظهٔ داده‌ساختار شما باید حداکثر از مرتبهٔ تعداد اعضای آن باشد.

پاسخ: پاسخ هر قسمت سؤال در ادامهٔ آن آمده است. توجه کنید که پاسخ سؤال‌ها یکتا نیست.

آ. **[۵ نمره]** سینا یک کتاب فروشی دارد، در این کتاب فروشی قیمت هر دو کتاب متفاوت است و برای مشتریان نیز تنها قیمت کتاب‌ها مهم است. سینا ابتدا تمامی کتاب‌های کتاب فروشی را به ترتیب قیمت وارد داده ساختار می‌کند. سینا ممکن است به دنبال یک کتاب با قیمت خاص باشد، یک کتاب بفروشد و یا کتاب‌های جدیدی به فروشگاه اضافه کند.

پاسخ: سینا به داده ساختاری نیاز دارد که درج، حذف و جستجو در آن در زمانی مناسب انجام شود. از آن‌جا که او از قیمت کتاب‌ها استفاده می‌کند و برای مشتریان نیز تنها قیمت اهمیت دارد، او می‌تواند از داده ساختار درخت جستجو استفاده کند و کتاب‌ها را بر اساس قیمت در این درخت قرار دهد. توجه کنیم از آن‌جا که ابتدا کتاب‌ها به ترتیب قیمت وارد می‌شوند، اگر از یک درخت جستجوی دودویی ساده استفاده کنیم باعث می‌شود ارتفاع درخت از مرتبه تعداد اعضا شود که مناسب نیست. بنابراین باید از درخت‌های دودویی جست‌وجوی متوازن مثل درخت قرمز–سیاه استفاده کنیم.

ب. **[۱۰ نمره]** یک استاد دانشگاه که تعداد بسیار زیادی دانشجو دارد، عملکرد کلاس خود را تنها بر اساس نمرات بیشینه و کمینه می‌سنجد. او می‌خواهد داده ساختاری داشته باشد که اضافه کردن نمره و حذف نمرات بیشینه (یا کمینه) در آن در زمان $\mathcal{O}(\log n)$ قابل انجام باشد، همچنین گزارش نمرات بیشینه و کمینه نیز در زمان $\Theta(۱)$ انجام شوند.

پاسخ: برای این سوال یک هرم بیشینه و یک هرم کمینه در نظر می‌گیریم و نمرات را در آن‌ها وارد می‌کنیم. در هر رأس علاوه بر نمره، اندیس همان نمره در هرم دیگر را نیز نگه می‌داریم. به صورت دقیق‌تر در هنگام درج ابتدا نمرهٔ جدید را به انتهای هر دو هرم اضافه می‌کنیم و اندیس هر کدام را در رأس دیگر ذخیره می‌کنیم. سپس ادامهٔ عملیات درج را انجام می‌دهیم که شامل تعداد جابه‌جایی رئوس است. در جابه‌جایی دو رأس، اندیس آن‌ها را در درخت دیگر به روزرسانی می‌کنیم. همچنین اگر بخواهیم نمرهٔ بیشینه (یا کمینه) را حذف کنیم، ابتدا اندیس متناظرش در هرم کمینه را نگه می‌داریم. سپس این نمره را در هرم بیشینه حذف می‌کنیم (در حین حذف وقتی جای نمرهٔ دیگری نیز تغییر کرد، اندیس متناظرش را در هرم دیگر به روز می‌کنیم). سپس نمره را از هرم کمینه حذف می‌کنیم (با توجه به این‌که اندیس آن را ذخیره کرده‌ایم). در حین حذف نیز اندیس‌های هرم دیگر را به روز نگه می‌داریم.

پ. **[۱۰ نمره]** یک استاد دانشگاه دیگر که او نیز تعداد بسیار زیادی دانشجو دارد، عملکرد کلاس خود را تنها بر اساس میانهٔ نمرات می‌سنجد. او می‌خواهد داده ساختاری داشته باشد که اضافه کردن نمره در آن در زمان $\mathcal{O}(\log n)$ قابل انجام باشد، همچنین گزارش میانه نیز از $\Theta(۱)$ باشد. توجه کنید اگر تعداد اعداد مجموعه فرد باشد، میانه برابر با عضو وسط مجموعه (در حالت مرتب شده) و اگر تعداد اعداد مجموعه فرد باشد برابر با میانگین دو عضو وسط مجموعه (در حالت مرتب شده) تعریف می‌شود.

پاسخ: برای این سوال نیز دو هرم کمینه و بیشینه در نظر می‌گیریم. طوری عمل می‌کنیم که اگر n نمره داشته باشیم، $\lfloor \frac{n}{۲} \rfloor$ نمره در هرم کمنیه و $\lceil \frac{n}{۲} \rceil$ نمره در هرم بیشینه قرار داشته باشد. همچنین تمامی نمرات هرم بیشینه از هرم کمینه کمتر مساوی باشند (معادلاً ریشهٔ هرم بیشینه کمتر مساوی ریشهٔ هرم کمینه باشد). واضح است که اگر این ساختار را داشته باشیم، گزارش میانه در زمان $\Theta(۱)$ انجام می‌شود (میانگین دو ریشه یا ریشهٔ هرم بیشینه). در ابتدا هنگامی که هیچ نمره‌ای نداریم این ویژگی برقرار است. پس کافی است تنها این ویژگی را هنگام درج کردن حفظ کنیم. فرض کنید می‌خواهیم عضو x را درج کنیم. اگر x بیشتر مساوی ریشه هرم بیشینه بود آن را در هرم کمینه و در غیر این صورت آن را در هرم بیشینه درج می‌کنیم. با این کار تمامی نمرات هرم بیشینه از هرم کمینه کمتر مساوی می‌مانند. تنها ممکن است رابطه تعداد اعضا برقرار نباشد. این اتفاق تنها در صورتی رخ می‌دهد که پس از درج هرم کمینه یک عضو بیشتر از هرم بیشینه داشته باشد. در این حالت کافی است ریشه هرم کمینه را حذف و آن را در هرم بیشینه درج کنیم. با این کار هم رابطه تعداد اعضا درست می‌شود و هم نمرات هرم بیشینه از هرم کمینه کمتر مساوی می‌مانند. پس توانستیم که این ویژگی را حفظ کنیم. واضح است که این کار حداکثر اندازه دو درج و یک حذف در هرم است. پس درج در این داده ساختار در زمان $\mathcal{O}(\log n)$ انجام می‌شود.

سؤال ۳. [۲۰ نمره]

آ. **[۵ نمره]** الگوریتمی طراحی کنید که k عنصر بزرگ‌تر یک آرایه را در $\mathcal{O}(n + k \log n)$ چاپ کند.

پاسخ: ابتدا در $O(n)$ آرایه را به هرم بیشینه تبدیل می کنیم. سپس در k مرحله، ریشهٔ هرم را چاپ کرده و آن را از هرم حذف می کنیم (عمل حذف در $O(\log n)$ انجام می شود).

ب. **[۵ نمره]** یک آرایه را در نظر بگیرید که هر عنصر آن با موقعیت خود در آرایهٔ مرتب شده، حداکثر به اندازه k فاصله دارد. الگوریتمی با زمان اجرای $\mathcal{O}(nk)$ طراحی کنید که این آرایه را مرتب کند.

پاسخ: با توجه به این که هر عنصر حداکثر به اندازه k با مکان نهایی خود فاصله دارد، با انجام مرتب‌سازی درجی، هر عنصر حداکثر k بار جابه‌جا می شود. پس با زمان اجرای $\mathcal{O}(nk)$ می توان این آرایه را مرتب کرد.

پ. **[۱۰ نمره]** آیا الگوریتم بهتری برای قسمت ب وجود دارد؟ اگر پاسخ مثبت است، روش خود را توضیح دهید.

پاسخ: بله. ابتدا k عنصر اول را درون یک هرم کمینه قرار می‌دهیم و همانند مرتب‌سازی هرمی ریشه را حذف کرده و آن را به آرایه اضافه می کنیم. سپس عنصر بعدی $(k+1)$ را در هرم درج می‌کنیم و این روند را ادامه می دهیم تا دیگر عنصری برای وارد کردن باقی نماند و در نتیجه با ادامه مرتب سازی هرمی می توانیم در زمان $\mathcal{O}(n \log k)$ به یک آرایهٔ مرتب شده برسیم.

سؤال ۴. **[۹ نمره]** با مرتب‌سازی سریع در کلاس آشنا شدید. پیچیدگی زمانی این الگوریتم را در حالات زیر بررسی و هر یک را اثبات کنید. قبل از پاسخ به این سؤال، نوع مرتب‌سازی سریع را (از سه نوع گفته شده در کلاس) به دلخواه انتخاب کنید و بر اساس آن به سؤالات پاسخ دهید. در پاسخ خود به صراحت اشاره کنید که کدام نوع مرتب‌سازی سریع را انتخاب کرده‌اید.

پاسخ: در این پاسخ نسخه‌ای از مرتب‌سازی سریع (غیرتصادفی) از دکتر قدسی در نظر گرفته شده است. در این نسخه عضر اول به عنوان لولا (محور) درنظر گرفته می‌شود. همچنین آرایه به دو قسمت (و نه سه قسمت) تقسیم می‌شود و این تقسیم به گونه‌ای پیاده‌سازی شده است که در صورتی که عناصر آرایه برابر باشند، آرایه به دو قسمت تقریباً مساوی تقسیم می‌شود.

آ. **[۳ نمره]** آرایهٔ ورودی مرتب باشد.

پاسخ: $O(n^2)$. هر تقسیم در $O(n)$ انجام می شود و عنصر محور هر دفعه کوچکترین عنصر است. پس در کل $O(n^2)$ طول می کشد تا اجرای الگوریتم تمام شود.

ب. **[۳ نمره]** همهٔ اعضای آرایهٔ ورودی برابر باشند.

پاسخ: $O(n \log n)$. هر تقسیم در $O(n)$ انجام می شود، اما محور هر دفعه به وسط زیر آرایهٔ مورد نظر می‌رود و آرایه را به دو بخش مساوی تقسیم می‌کند.

پ. **[۳ نمره]** آرایهٔ ورودی به صورت نزولی مرتب شده باشد.

پاسخ: $O(n^2)$. آرایه در هر مرحله به دو قسمت به اندازه های ۱ و $n-1$ تقسیم می شود و چون زمان اجرای تقسیم $O(n)$ است، زمان اجرای الگوریتم در کل برابر است با $O(n^2)$.

سؤال ۵. **[۱۰ نمره]** نشان دهید در یک درخت دودویی جستجو از هر عنصری شروع کنیم می توانیم k عنصر بعدی آن را در زمان $\mathcal{O}(h+k)$ پیدا کنیم که h ارتفاع درخت است.

پاسخ: رأس ابتدایی را s و رأس kاُم بعدی آن را t می‌نامیم. مسیر یکتایی که بین این دو راس در درخت وجود دارد را p و پایین‌ترین جد مشترک آن‌ها را l بنامید. طول p حداکثر برابر $2h$ است، زیرا این مسیر از ترکیب دو مسیر، یکی از s و l و دیگری از l به t تشکیل شده است. برای یافتن جواب علاوه بر رئوس p، تنها زیردرخت‌هایی از رئوس مسیر(به جز s و l و t)، و به صورت کامل دیده می‌شوند (بدین معنی که در آن‌ها رأس اضافه‌ای که داخل جواب نباشد دیده نمی‌شود). بنابراین هر راس از این زیردرخت‌ها، یکی از k راس مدنظر ماست، که دقیقا هم یک‌بار دیده می شود. پس در کل به اندازه $2h+k=\mathcal{O}(h+k)$ هزینه می‌کنیم.

سؤال ۶. [۱۰ نمره]

نشان دهید دو درخت قرمز‌ـسیاه دلخواه n رأسی را می‌توان با حداکثر $2n - 2$ دوران به هم تبدیل کرد. (فرض کنید رنگ رأس‌ها برای ما مهم نیستند و رأس‌های nil را نیز درنظر نگیرید)

پاسخ: برای اثبات این سوال کافی است برای هر کدام از درخت‌ها، با $n - 1$ دوران به یک درخت یکسان برسیم. در این صورت اگر با $n - 1$ حرکت درخت اول به درخت ذکر شده برویم سپس $n - 1$ حرکت درخت دوم به این درخت را به صورت معکوس انجام دهیم و به جای دوران به چپ، دوران به راست از همان رأس‌ها انجام دهیم و به جای دوران به راست، دوران به چپ انجام دهیم، درخت ذکر شده به درخت دوم تبدیل می‌شود. بنابراین با حداکثر $2n - 2$ بار دوران از درخت اول به درخت دوم می‌رسیم. حال نشان می‌دهیم درخت مسیر خاصیت گفته شده را دارد و با $n - 1$ دوران می‌توانیم هر درختی را به آن تبدیل کنیم. میسری را در نظر میگیریم که رأس ریشه را دارد و هم چنین هر راسی که در این مسیر آمده باشد، فرزند چپش هم (در صورت وجود) در این مسیر آمده است. اسم این مسیر را M می‌گذاریم و طول اولیهٔ آن را k در نظر می‌گیریم. هر بار یک رأس از این مسیر که فرزند راست دارد را می‌گیریم و دوران چپ روی آن رأس اعمال می‌کنیم. در این صورت می‌توان مشاهده نمود که طول مسیر M یکی زیاد می‌شود و برابر $k + 1$ می‌شود. این الگوریتم را آنقدر تکرار می‌کنیم تا برای هیچ راسی از M فرزند راست باقی نمانده باشد و در نتیجه درختمان به درخت مسیر که مورد نظر بود تبدیل شود. از آنجایی که با هر بار انجام یک دوران، به طول M یکی افزوده می‌شود و در نهایت هم برابر $n - 1$ می‌شود، پس تعداد عملیات انجام شده از $n - 1$ بیشتر نیست. بنابراین حکم مورد نظر اثبات می‌شود.

سؤال ۷. [۲۶ نمره]

آ. **[۱۶ نمره]** یک درخت دودویی جستجوی کامل داریم. الگوریتمی با زمان اجرای $\mathcal{O}(n)$ طراحی کنید که این درخت دودویی جستجو را به هرم بیشینه‌ای تبدیل کند که گره‌های زیردرخت راست هر رأس آن از گره‌های زیردرخت سمت چپش بزرگ‌تر باشند (در هر گام می توانیم جای دو رأس را با هم عوض کنیم).

پاسخ: واضح است که پیمایش پس‌ترتیب هرم بیشینه عناصر را به صورت مرتب شده به ما تحویل می‌دهد. در مورد درخت دودویی جستجو هم می‌دانیم که پیمایش میان‌ترتیب آن عناصر را به صورت مرتب‌شده به ما می‌دهد. پس پیمایش میان‌ترتیب د.د.ج با پیمایش پس‌ترتیب هرم یکسان است. در نتیجه می‌توانیم مقدار هر رأس هرم بیشینه را در $O(n)$ به‌دست آوریم (چون پیمایش یک درخت در زمان $O(n)$ انجام می شود) و بعد از آن هم به راحتی با عوض کردن هر رأس با رأس مورد نظر مسئله حل می‌شود.

ب. **[۱۰ نمره]** ثابت کنید برای تبدیل یک درخت دودویی متوازن که درخت آن تقریباً کامل نیز هست (مثل هرم) به هرم بیشینه‌ای مشابه قسمت آ، الگوریتمی بهتر از $\mathcal{O}(n \log n)$ وجود ندارد (مانند قسمت قبل در هر مرحله تنها می‌توانیم جای دو رأس را با هم عوض کنیم).

پاسخ: پیمایش پس ترتیب هرم را در نظر بگیرید و آن را A بنامید. عوض کردن جای دو رأس مثل این است که جای دو عدد را در A تغییر بدهیم. برای این‌که شرط مسئله محقق شود باید A مرتب شود در حالی که در ابتدا اعداد موجود در آرایهٔ A هیچ شرط خاصی ندارند. می‌دانیم این کار در زمانی کمتر از $O(n \log n)$ میسر نیست.

سؤال ۸. [۲۰ نمره] زوج مرتب های مرتب

فرض کنید ۱,۰۰۰,۰۰۰ زوج مرتب مثل (a, b) داریم که باید آن ها را طبق قانون زیر مرتب کنیم:

$$(a_1, b_1) < (a_2, b_2) \iff (a_1 < b_1) \vee \big((a_1 = b_1) \wedge (b_1 < b_2)\big)$$

ورودی: شامل ۱,۰۰۰,۰۰۰ خط است که در هر خط زوج مرتب (a, b) به ترتیب به کاربر داده می شود.

$$1 \le a < 100\,000$$
$$1 \le b \le 1\,000\,000\,000\,000\,000$$

خروجی: باید زوج مرتب‌ها پس از مرتب کردن باشد (در هر خط یک زوج مرتب).

نکات: در این مسئله شما تنها مجاز به استفاده از زبان پایتون و جاوا هستید.

ورودی دقیقا ۱,۰۰۰,۰۰۰ زوج مرتب دارد اما به علت محدودیت صورت سؤال، مثال زیر برای ۱۰ زوج مرتب است.

ورودی و خروجی نمونه:

input	output
13241 323294848907982	13241 152840012736194
97255 553012130624557	13241 241855619495066
13241 876302146194655	13241 323294848907982
33481 299942243878034	13241 876302146194655
97255 858692723362742	33135 358707532837541
33135 358707532837541	33481 299942243878034
13241 241855619495066	49483 395372516597937
63663 186195109346510	63663 186195109346510
49483 395372516597937	97255 553012130624557
13241 152840012736194	97255 858692723362742

پاسخ: مقدار a محدود است، پس می‌توانیم بر اساس زوج مرتب‌ها را در براساس aهایشان در سطل‌های مختلف بریزیم و اعداد داخل هر سطل را با یک الگوریتم مرتب سازی دلخواه مرتب کنیم (مثل مرتب‌سازی شمارشی یا ادغامی). کد آقای وحید زهتاب با نام `ordered-pair.py` به عنوان نمونه در پوشه مربوطه قرار داده شده.

سؤال ۹. [۲۰ نمره] نابه‌جایی با AVL

در آرایهٔ $A = (a_۱, a_۲, ..., a_n)$ به زوج i و j که $i > j$ و $a_i < a_j$ یک نابه‌جایی می‌گویند. یک آرایهٔ n عضوی به شما داده می‌شود و شما باید برای هر زیردنبالهٔ kتایی متوالی آن تعداد نابه‌جایی‌ها را به دست آورید و سپس مجموع این اعداد را حساب کنید. برای حساب کردن این مجموع شما باید از درخت AVL استفاده کنید.

ورودی: در خط اول به ترتیب اعداد n و k به شما داده می‌شود. در خط بعد n عدد به ترتیب به شما داده می‌شود.

خروجی: خروجی شما باید مجموع تعداد نابه‌جایی‌ها به پیمانه‌ی $۱۰^۹ + ۷$ باشد.

نکات: در صورتی که از درخت AVL در پیاده‌سازی خود استفاده نکنید نمره‌ای به شما تعلق نخواهد گرفت.

ورودی و خروجی‌های نمونه:

input	output
6 2 1 2 3 4 5 6	0
4 3 1 6 7 2	2
10 5 15 51 44 44 76 50 29 88 48 50	25
8 4 12 3 14 8 15 1 4 22	14

پاسخ: درخت AVLای که برای حل این سؤال استفاده می‌کنیم، یک درخت مرتبه آماری با قابلیت درج و حذف است (مشابه آن‌چه برای محاسبهٔ مرتبه (rank) در جلسهٔ ششم درس مطرح شد). این درخت برای هر رأس، تعداد رئوس زیردرخت آن رأس را ذخیره می‌کند و با استفاده از آن جایگاه هر عنصر را در آرایهٔ مرتب شده به دست می‌آورد. ابتدا با k عنصر اول این درخت را می‌سازیم و تعداد نابه‌جایی‌های آن را حساب می‌کنیم. تعداد نابه‌جایی‌هایی که هر عنصر با عناصر قبل از آن ایجاد می‌کند برابر تعداد عناصری است که هنگام درج آن عنصر در درخت از آن بزرگ‌تر هستند. حال می‌خواهیم تعداد نابه‌جایی‌های k عنصر بعدی را پیدا کنیم. ابتدا عنصر اول kتایی قبلی را حذف می‌کنیم و تعداد نابه‌جایی‌هایی که ایجاد می‌کرد را از تعداد نابه‌جایی‌های kتایی کم می‌کنیم تا تعداد نابه‌جایی‌های $k - ۱$ تایی به دست آید. سپس عنصر جدید را درج می‌کنیم و تعداد نابه‌جایی‌هایی که ایجاد می‌کند را به تعداد نابه‌جایی‌های $k - ۱$تایی اضافه می‌کنیم تا تعداد نابه‌جایی‌های k تایی جدید به دست آید. به این صورت می‌توان جمع تعداد نابه‌جایی‌های kتایی‌ها را در زمان $O(n \log k)$ به دست آورد. کد ضمیمه شده برای این سوال پاسخ ارسالی آقای وحید زهتاب می‌باشد که در فایل `avl.py` موجود است.

سؤال ۱۰. [۲۰ نمره] تذکر: در این سؤال استفاده از هیچ یک از متدهای مرتب‌سازی مجاز نیست.

الگوریتمی پیاده‌سازی کنید که با دریافت k آرایهٔ مرتب به طول n، آرایهٔ مرتب شده حاصل از ادغام این آرایه‌ها را در خروجی چاپ کند.

ورودی:

در خط اول ورودی عدد n و در خط دوم عدد k داده می‌شود. در ادامه اعداد هر آرایه در سطرهای متمایز قرار دارند.

خروجی:

آرایهٔ مرتب شده را در خروجی چاپ کنید.

ورودی و خروجی نمونه:

ورودی	خروجی
4 3 7 7 8 9 4 4 5 6 1 1 2 3	1 1 2 3 4 4 5 6 7 7 8 9

پاسخ: تمام سرلیست‌ها را در یک هرم کمینه درج می کنیم. سپس در هر مرحله عنصر کمینه را حذف و عنصری که در ادامهٔ لیست ظاهر می‌شود را در هرم درج می‌کنیم. این کار را تا به پایان رسیدن همهٔ آرایه‌ها ادامه می‌دهیم. کد این سؤال هم کد خانم مهسا دلیرروی فرد است که در فایل `merge.py` ضمیمه شده.

موفق باشید